18/03/26

→ Existem otimizações possíveis mas não necessariamente viável

→ análise de corretude + verificar se o algo. entrega o que deveria

Corretude
→ Técnicas: Indução, Contradição, Invariantes de laço

Complexidade
→ Critérios → Tempo, Comparações, Troca

Corretude → demonstrar formalmente que: para toda instância do problema o algoritmo produz uma saída correta

→ Não existe meio certo em algoritmos

→ Invariante de laço / loop invariant
↳ dado um resultado S' difícil resultado S_i menor e provar cada uma
↳ Se todo S_i está correto S também está

- **Algumas etapas:** Critério de Parada, Inicialização, Manutenção e Término

Etapa:

1. Vr se S é verdadeiro
2. Se S_{i-1} é verdadeiro (antes da i) verifica S_i
3. S_K / qualquer dem implica que S também é verdadeiro

Insertion Sort

↳ dois arrays um ordenado o outro não

↳ pega um elemento por vez do não ordenado e bota ele na posição certa no ordenado

for i in range(1, len(lista)):
    for j in range(i, 0, -1):
        if lista[j] < lista[j-1]:
            lista[j], lista[j-1] = lista[j-1], lista[j]

$O(n^2)$

- **Etapa 1: Inicialização** (*S* inicial é verdadeira antes que o laço seja iniciado)
  - **Invariante Principal (i1):** subvetor **A[1…*j*-1]** e **estado ordenado;**
    - Linha 1 (primeira iteração): subvetor está com **A[1] e ordenado;**
  - **Laço invariante válido para S_{i-1}.**

- **Etapa 2: Manutenção** (S_i será verdadeiro depois que a interação *i* terminar)
  - **Invariante de Manutenção (i2)**
    - Linhas 1 a 8: **cada iteração mantém laço invariante**: subvetor **A[1…*j*-1] ordenado**
      - Linha 3: **Inserção de A[*j*] na posição correta**
  - **S_i verdadeiro após interação *i* finalizada:** Subvetor **A[1…*j*-1] ordenado**

- **Invariantes Auxiliares (Expansão de (i2))**
  - **Invariantes no início da Linha 5:**
    - **(i3):** subvetor **A[1…*i*] (ordenado)** e subvetor **A[*i* + 2…*j*] (não ordenado)**
    - **(i4): A[1…*i*]** e **A[*i* + 2…*j*]** são crescentes (limite é *comprimento* [A])
    - **(i5): A[1…*i*] ≤ A[*i* + 2…*j*]**
    - **(i6): A[*i* + 2…*j*] > *chave* (*supostamente não ordenado*)**

- **Etapa 3: Término** (S_k implica que a afirmação *S* é verdadeira)
  - Linhas 1 a 8: **cada iteração mantém laço invariante**: subvetor **A[1…*j*-1] ordenado;**
  - Última iteração: ***j* ← *n*;**
  - Corretude do algoritmo está garantida: **A[1…*n*] ou S_k** ordenado ⊢ **A[1…*j*-1] ordenado.**

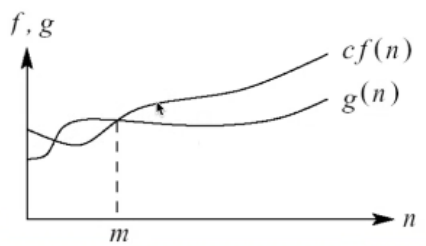
garante que o algo. até aqui

Complexidade → analisa complexidade
× compl. assintótica
↳ o que faz o algoritmo
refere-se ao número de
n
↳ o tamanho de n que
faz ele refere depende do
algoritmo

Comportamento assintótico de f(n)
↳ representa o limite do
comportamento do custo
quando n cresce
↳ ajuda a comparar funções
(algoritmos)
↳ não indica o máximo, mas
mostra o comportamento ao
longo do tamanho de n

Dominação assintótica → uma função
domina a outra

- $f(n)$ **domina assintoticamente** outra função $g(n)$
  - se há duas constantes positivas c e m tais que, para $n \geq m$ tem-se $|g(n)| \leq |c\, f(n)|$



f(n) domina g(n)

o ponto m é
o ponto que f(n)
domina

Notação

→ Big O → marca o limite assintótico superior

↳ pior caso

↳ permite dizer que g(n) é ≤ f(n) a menos de uma constante c

· g(n) domina

- Utiliza-se: **g(n) = O(f(n))** para representar:

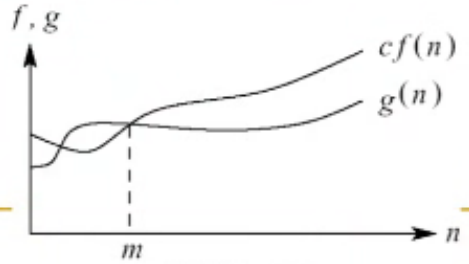
  - *g*(*n*) é da **ordem de no máximo** *f*(*n*)

    - Exemplo: Um programa com tempo de execução (*T*(*n*)) é *O*(n^2)

      - Interpretação: há constantes *c* e *m* tais que, para valores $n \geq m$, $T(n) \leq c.n^2$

} caso extremo

← no pior caso T(n) cresce em n^2



Classe de comportamento assintótico

O(1) ⇒ constante. Não depende de n

⇒ executa um número fixo de vezes

$O(\log n)$ => particionamento do problema

$O(n)$ => aumento proporcional com n
→ mais buscado

$O(n \log n)$ => Transforma em pequenos dividir pontos
=> proporcional a n

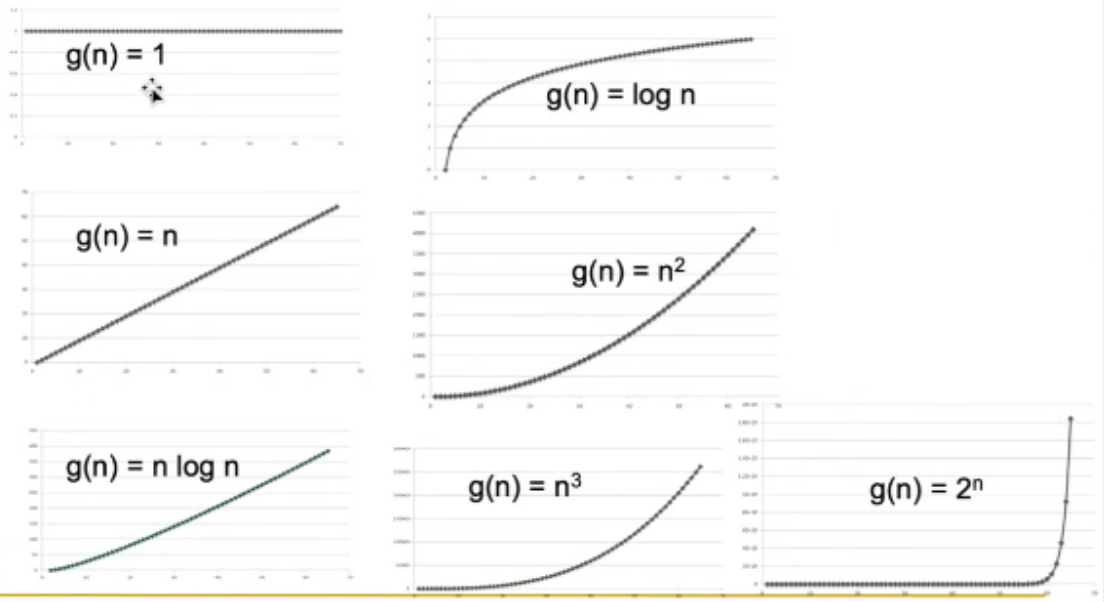
$O(n^2)$ => pesado, laço dentro do outro x4

$O(n^3)$ => problema pequeno, laço aninhado (produto de matrizes) x8

$O(2^n)$ => exponencial, ~~difícil~~ mais
Força bruta, Não é prático

$O(n!)$ => Força bruta, o pior.
Não prático

- **Gráficos de Comportamento Assintótico**



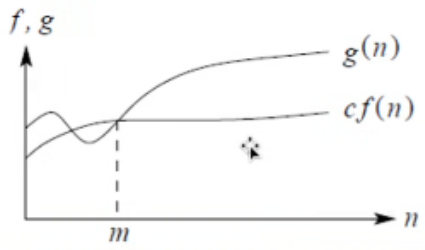
Big Ω → pouco utilizado
↳ limite assintótico inferior
↳ f(n) cresce a uma taxa ≥ m

Para problemas não mapeados precisamos ir analisando part por part

- $f(n)$ $\Omega(g(n))$: $f(n)$ é ômega de $g(n)$

↳ Em geral queremos $\Omega(1)$

- Exemplo: Programa com tempo de execução ($T(n)$) é $\Omega(n^2)$
  - Interpretação: há constantes c e m tais que, para valores $n \geq m$, $T(n) \geq c.n^2$



$g(n) = \Omega(f(n))$

g domina

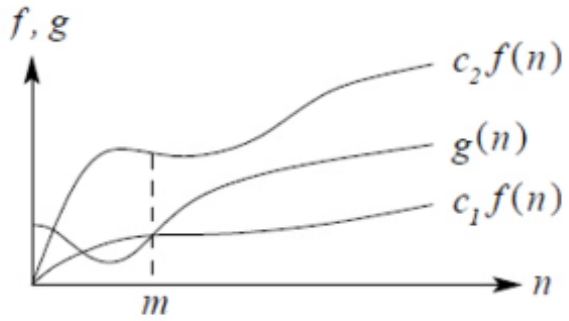
Complexidade de Algoritmos

- **Notação Θ (Theta)**
  - Utilizada para dar um **limite assintótico restrito**
    - $f(n)\ \Theta(g(n))$: $f(n)$ é theta de $g(n)$
      - Permite afirmar que duas funções **crescem na mesma proporção (taxa)**, até fatores constantes

função entre dois limites

- Utilizamos $\boldsymbol{g(n) = \Theta(f(n))}$ para expressar:
  - o valor de $\boldsymbol{g(n)}$ está sobre ou acima de $c1f(n)$ e sobre ou abaixo de $c2f(n)$.

- Exemplo: Programa com tempo de execução ($T(n)$) é $\Theta(n^2)$
  - Interpretação: há constantes positivas c_1, c_2 e m tais que $0 \leq c_1 n^2 < g(n) < c_2 n^2$ para todo $n \geq m$.



$\in g(n) = \Theta(f(n))$

mult. por constante

Técnica - análise de complexidade

1. Operação básica de notação
   - determina tempo de execução
   - ordem de tempo
   - não pensa nos valores constantes

Complexidade de Algoritmos - Técnicas de Análise

- **Operação Comumente Utilizadas**
  - Exemplos com notação O

	$f(n)$	$\Rightarrow O(f(n))$	
	$c \times O(f(n))$	$\Rightarrow O(f(n))$	$c = constante$
	$O(f(n)) + O(f(n))$	$\Rightarrow O(f(n))$	
	$O(O(f(n))$	$\Rightarrow O(f(n))$	
Regra da soma	$O(f(n)) + O(g(n))$	$\Rightarrow O(max(f(n), g(n)))$	
Regra do produto	$O(f(n))O(g(n))$	$\Rightarrow O(f(n)g(n))$	
	$f(n)O(g(n))$	$\Rightarrow O(f(n)g(n))$	

Regras utilizadas para calcular o tempo de execução de uma sequência de blocos de códigos

Notação *O*: Um Exemplo Téorico

- Considere três partes de um algoritmo com os tempos:

$$O(n),\ O(n^2) \text{ e } O(n \log n)$$

- Passo 1: Analisar o tempo de execução dos dois primeiros

$O(max(n, n^2))$, que é $O(n^2)$

- Passo 2: Analisar o tempo de execução de todos os três

$$O(max(n^2, n \log n)) \longrightarrow O(n^2)$$

Técnicas de Análise

1. **Iniciar pelo laço mais interno**

   a) Avaliar comandos internos

   b) Linha 4: *O*(1)

   c) Assumir pior caso: linha 5 sempre executada, *O*(1)

   d) ***O*(max(1,1))= *O*(1)**

```
int Max (int *A, int n)
{
[redacted]
3.      for (i=1;i<n;i++)
        {
4.          if (max<A[i])
5.            max=A[i];
[redacted]
```

Começa a análise internamente (pela indentação)

Complexidade de Algoritmos - Técnicas de Análise

2. **Avaliar linha 3**

a) Tempo para incrementar índice do Laço: $O(1)$

b) Tempo para avaliar a condição do Laço: $O(1)$ e instanciar i $O(1)$

c) $O(\max(1,1,1,1)) = O(1)$
- Considerar resultado da etapa 1d

d) Considerar o número de iterações (Regra do produto)
- $O((n-1) \times 1) = O(n-1)$

```
int Max (int *A, int n)
{
[redacted]
3.      for (i=1;i<n;i++)
[redacted]
```

iteração x o interno

Complexidade de Algoritmos - Técnicas de Análise

3. **Avaliar comandos fora do laço**

a) Linha 1: $O(1)$
b) Linha 2: $O(1)$
c) Linha 6: $O(1)$

d) $O(\max(1,1,n-1,1)) = O(n-1)$

Importante:
- $O(n-1)$ não é uma representação válida.
  - $O(\max(1,1,n,1)) = O(n)$
  - Logo, tem-se $O(n)$.

```
int Max (int *A, int n)
{
1.      int i, max;
2.      max=A[0];
[redacted]
6.      return max;
}
```

considero o resultado o resto constante

O laço i o mais pesado

Resumo

- Comandos:
  - Atribuição, leitura, escrita, decisão, chamadas e/ou de retornos de funções ou métodos: $O(1)$;
  - Sequência de commandos;
    - Determinar o **maior tempo de execução** de qualquer comando da sequência (**Regra da soma**);
  - Laço
    - Somar o tempo de execução do corpo do laço mais o tempo de avaliar a condição para terminação (geralmente O(1)), multiplicado pelo número de iterações (**Regra do produto**);

↳ A análise inicia com frequentemente que não chamam outras análises

2- Identificar polinômio

- Identificação dos Polinômios

↙ Carga para cada comando

Algoritmo *arrayMax* (A, n)	Operações
$Max \leftarrow A[0]$;	1
for $(i \leftarrow 1;\ i < n;\ i{+}{+})$	$(n-1)+1$
{ **if** $(A[i] > Max)$ **then**	$(n-1)$
$Max \leftarrow A[i]$;	$(n-1)$
}	
return Max	1
Total	$3n + 0$

→ coeficiente angular

Logo, o polinômio $3n$ é $O(n)$, para $c = 3$ e $m = 0$.

→ Por certo lado ⊕ pode ser visto
como a média está $O(1)$ a
$SL(h)$, mas não é bem certo.